Smart Fan Speed Controller

A Fuzzy Logic and Genetic Programming PBL Submission

L.D. College of Engineering, Ahmedabad
Chemical Engineering, Division A
8 October 2026

Group member	Enrollment number
Kirtan Siddhapura	240280105071
Mahesh Bhaliya	240280105005
Manav Chauhan	240280105008
Haresh Raval	240280105065

Project scope

A reproducible software simulation that maps temperature and relative humidity to a fan speed command. Nine interpretable Mamdani rules form the main controller. A separate tree genetic programming experiment approximates its simulated response.

No physical fan, sensor trial, measured energy saving, or human comfort study is claimed. Membership limits and rules are classroom design assumptions.

Submission contents: 12-slide presentation, report, Python source and notebook, narrated MP4, repository documentation, and publication copy. Public links must be added after upload.

1. Problem

Manual fan speed adjustment requires users to react to changing room conditions. A fixed temperature threshold can change the command abruptly and ignores humidity. Our simple problem is to generate a gradual, interpretable fan command from two environmental inputs.

A fan moves air. This model does not predict a decrease in room temperature or remove moisture. Humidity acts only as a heuristic cue for a higher airflow request.

Use case

A classroom or hostel-room demonstration using simulated temperature and humidity readings. For future hardware, a sensor would provide inputs and an appropriate isolated motor driver would translate a command to actuator input. No mains wiring is part of this submission.

2. Objectives

Define overlapping low, medium and high fuzzy sets. Encode nine understandable rules. Implement min-max Mamdani inference and centroid defuzzification. Check boundary coverage and invalid inputs. Explain example outputs and limitations. Demonstrate actual genetic programming through expression-tree evolution.

3. System Design

Variable	Universe	Meaning
Temperature	15 to 40 C	Input room temperature
Humidity	20 to 90 %RH	Input relative humidity
Fan command	0 to 100%	Normalized requested speed, not measured RPM

Processing sequence

Validate inputs, fuzzify both readings, calculate nine rule strengths using minimum, clip each consequent, aggregate using maximum, and calculate the centroid on a 0.1 percentage-point output grid.

Triangular membership specification

Variable	Low (a,b,c)	Medium (a,b,c)	High (a,b,c)
Temperature	(15,15,27)	(20,28,36)	(29,40,40)
Humidity	(20,20,55)	(35,60,85)	(65,90,90)
Fan command	(0,0,50)	(25,50,75)	(50,100,100)

For a < b < c, membership is max(0, min((x-a)/(b-a), (c-x)/(c-b))). Repeated endpoints form shoulders. Membership equals 1 at the repeated endpoint, and 0 outside the universe.

These design parameters cover every input in the specified domains. They are illustrative rather than calibrated thermal comfort thresholds.

4. Membership Functions

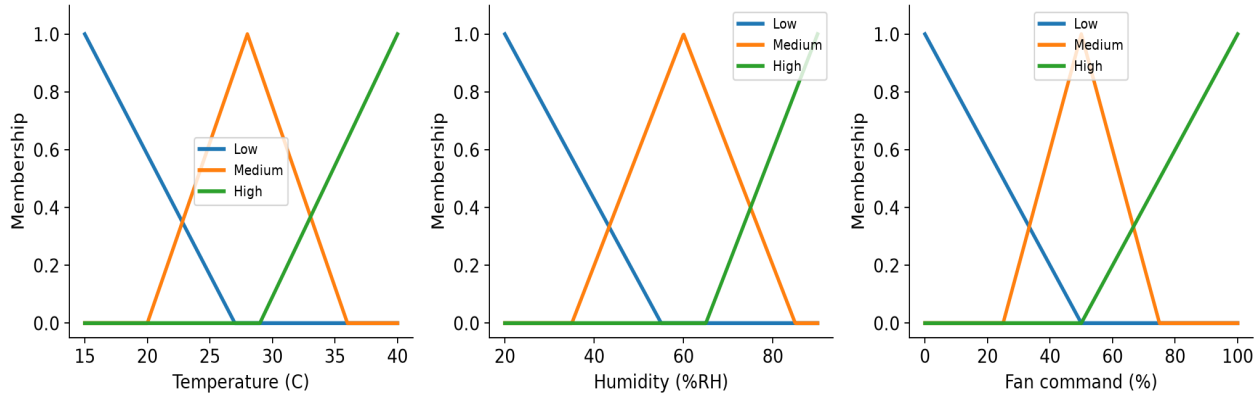


Figure 1. Overlapping memberships allow a reading to activate multiple rules. Low and high sets use shoulders to cover domain endpoints.

At 30 C, temperature memberships are Low = 0, Medium = 0.750, High = 0.091. At 80% RH, humidity memberships are Low = 0, Medium = 0.200, High = 0.600. These degrees describe membership in sets and are not probabilities.

5. Rules

Temperature / Humidity	Low	Medium	High
Low	Low	Low	Medium
Medium	Low	Medium	High
High	High	High	High

Each cell is one rule. For example: IF temperature is Medium AND humidity is High THEN fan command is High. AND uses minimum. All rules have equal weight.

Rationale: cooler conditions usually request less airflow, higher humidity can increase the request, and a high temperature requests high airflow at all humidity levels.

6. Implementation

The supplied smart_fan.py implements inference directly in NumPy so the main controller has no dependency on scikit-fuzzy. It follows the same Mamdani min-max structure shown in the official scikit-fuzzy control example. Plotting requires matplotlib. The notebook includes all source code and can run independently.

Inference equations

Rule strength alpha(i,j) = min(mu_temperature_i, mu_humidity_j).
Aggregate A(z) = max over rules of min(alpha(i,j), mu_output_consequent(z)).
Command = integral z A(z) dz / integral A(z) dz.
Numerical integration uses NumPy trapezoidal integration.

Worked inference: 30 C and 80% RH

The four active rules have strengths 0.200 (Medium/Medium), 0.600 (Medium/High), 0.091 (High/Medium), and 0.091 (High/High). Aggregation combines Medium clipped at 0.200 with High clipped at 0.600. The numerical centroid is 73.113%.

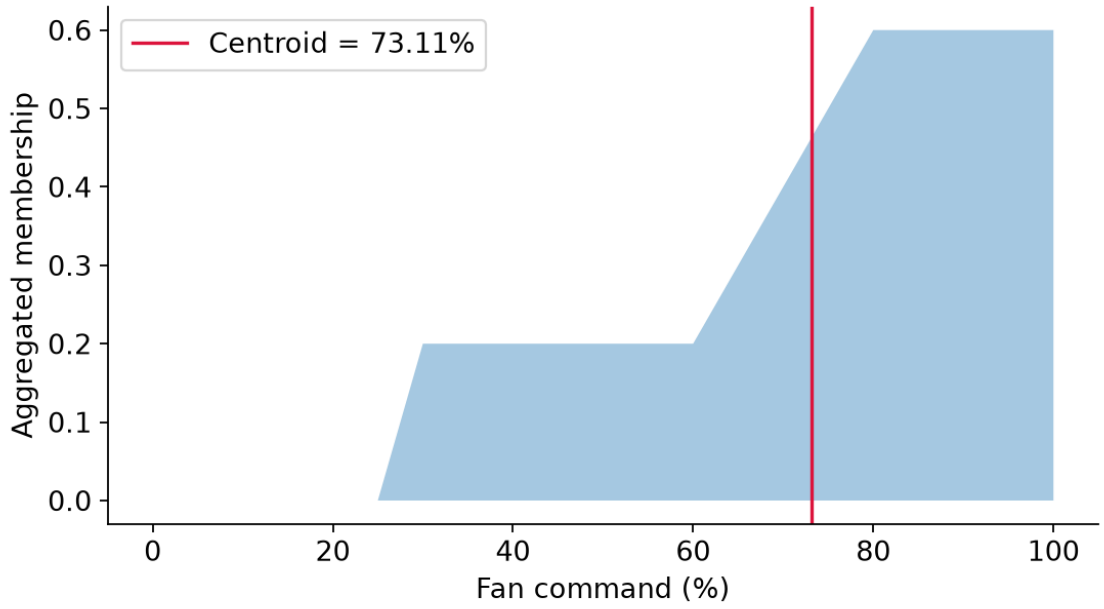


Figure 2. The shaded output membership and its centroid.

7. Results

Temperature (C)	Humidity (%RH)	Command (%)
15	20	16.667
22	40	29.848
28	60	50.000
30	80	73.113
35	50	67.328
40	90	83.333

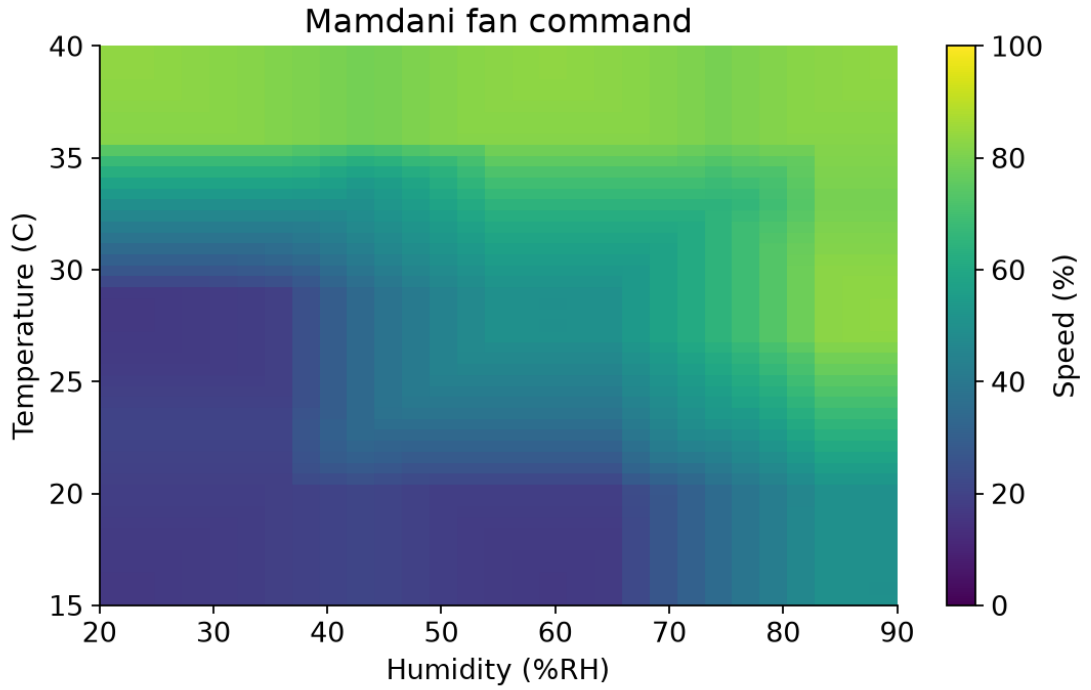


Figure 3. The simulated response surface. Every value comes from the same controller used for the example table.

8. Analysis and Verification

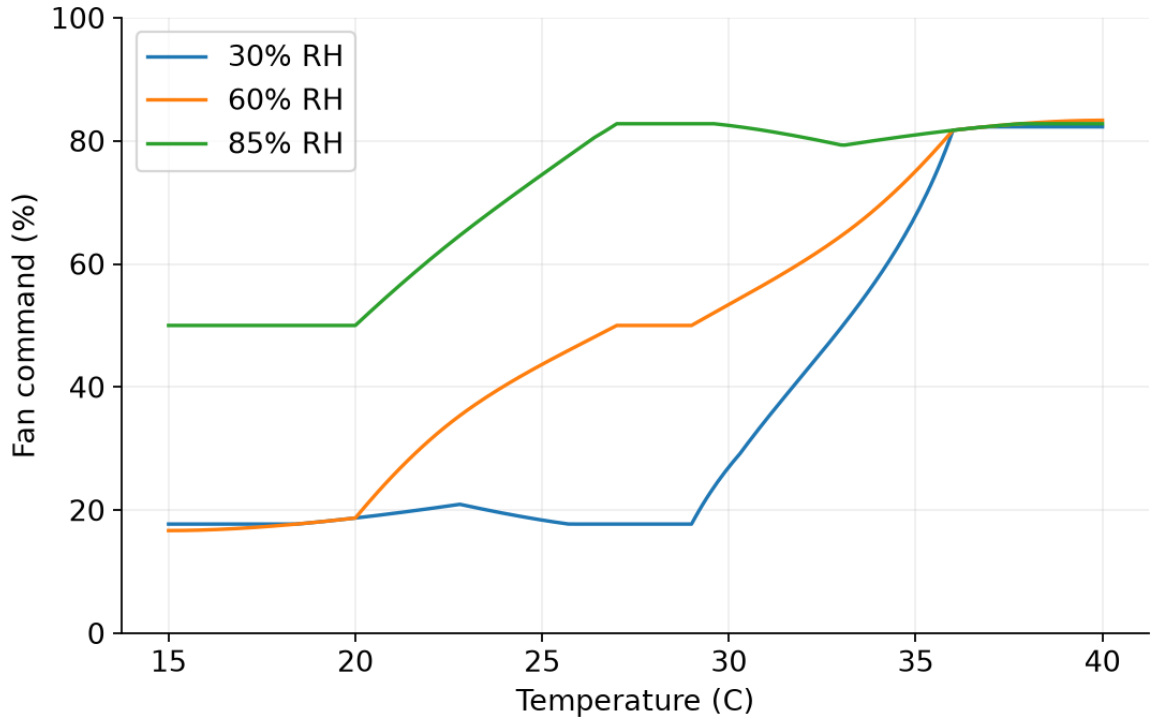


Figure 4. Temperature sweeps at three fixed humidity levels.

All 1479 grid points produced finite commands between 16.667% and 83.333%. Invalid and nonfinite inputs raised ValueError. The GP run uses a fixed seed for repeatability.

The low and high centroids are about 16.667% and 83.333%, respectively. A fully active shoulder set has its centroid inside its support, so a 100% high membership does not generate a 100% command.

The response is continuous but not strictly monotonic. On a 0.5 C / 2.5% RH grid, the largest adjacent temperature decrease is 1.358 percentage points and the largest humidity decrease is 2.387 points. Changing clipping heights and overlaps can move the centroid backward locally. These are measured model limitations, not numerical test failures.

A future design could enforce monotonicity with calibrated singleton consequents or constrained rule tuning. This submission retains the transparent Mamdani model and reports the actual behavior. No power model is available, so energy savings cannot be calculated.

9. Genetic Programming Experiment

Genetic programming evolves executable expression trees. This differs from a genetic algorithm that tunes a fixed vector of numerical parameters. Here GP learns an approximate mapping from normalized inputs to the fuzzy output, using synthetic labels from the main controller. It is a separate educational experiment and does not replace the main controller.

Setting	Value
Inputs	t=(T-15)/25, h=(RH-20)/70
Terminals	t, h, 0.25, 0.5, 1.0, -0.5
Primitives	addition, subtraction, multiplication, min, max
Data	11 x 11 grid: 85 training, 36 held-out test points
Evolution	100 trees, 40 generations, seed 71
Selection / variation	Tournament size 3, crossover 0.8, mutation 0.2, 2 elites
Complexity control	Maximum depth 6; MSE + 0.00005 x node count
Output	100 x clip(tree(t,h), 0, 1)

Training RMSE = 9.215 percentage points. Held-out test RMSE = 9.431 percentage points. The test set is never used for selection.

Best evolved expression (unsimplified)

max(max(((h * 0.25) - (0.25 - t)), (-0.5 + (((0.25 * t) - (0.25 - 0.25)) + h))), max(0.25, -0.5))

Algebraically, this equals max(t + 0.25h - 0.25, 0.25t + h - 0.5, 0.25), followed by clipping. All claims concern approximation of this synthetic target, not human comfort prediction.

10. GP Results, Conclusions and References

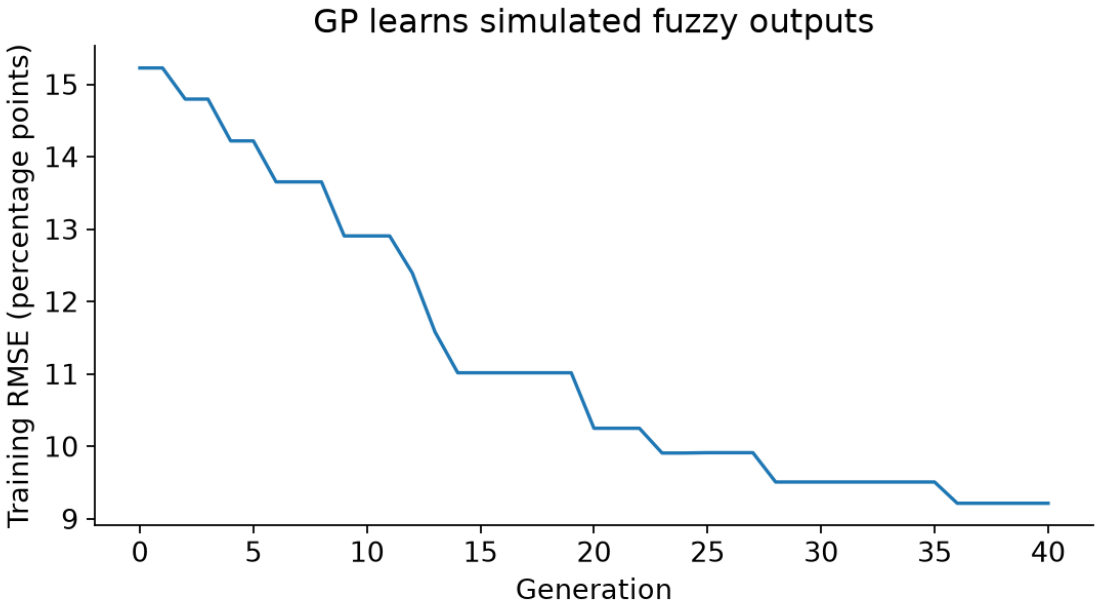


Figure 5. Training RMSE across generations. Selection includes a size penalty, so unpenalized RMSE need not decrease in every generation.

Conclusion

The nine-rule controller works across the declared input domain and makes its reasoning visible. Temperature and humidity influence the command without a single hard switch. Centroid behavior creates local nonmonotonic regions and caps the command range. The GP model provides a compact approximation with measurable error. Hardware calibration and user comfort testing remain future work.

Suggested group workflow

Review the source together, reproduce the results, assign presentation speaking parts, and publish only after each member confirms the work. Member-specific authorship or completed tasks are not asserted here.

References

1. scikit-fuzzy, Fuzzy Control Systems: The Tipping Problem. https://scikit-fuzzy.github.io/scikit-fuzzy/auto_examples/plot_tipping_problem_newapi.html
2. DEAP, Genetic Programming tutorial. https://deap.readthedocs.io/en/stable/tutorials/advanced/gp.html
3. DEAP, Symbolic Regression example. https://deap.readthedocs.io/en/stable/examples/gp_symbreg.html
Accessed 8 October 2026. Our membership parameters, rules, dataset and results are project-specific.